

Introduction to Python random library

Abdesselam Filali

infos@filali.net

https://github.com/Abdou-fi/python_libraries

Mai 02, 2026

This is a Jupyter notebook converted to L^AT_EX using the nbconvert tool. It contains explanation, code snippets and outputs related to the Python *random* library. For any question or remark please email me or send a request through my github.

1 Random Module in Python

1.1 What is the Random Module?

The random module in Python is like a magic toolbox that helps you work with randomness! It allows you to create random numbers, pick random items from a list, or even shuffle things around. Think of it like this: If you want to roll a virtual dice, pick a winner from a list of names, or create a surprise by mixing things up, the random module is there to help! You just tell it what kind of random thing you need, and it does the job for you.

```
[33]: # Example usage of the random module in Python.  
# This will print a random number between 0 and 10 every  
# time you run the code!  
  
import random  
# pick a random number between 0 and 10  
random_number = random.randint(0, 10)  
print(random_number)
```

1

1.2 Why Use the random Module?

The random module is super handy whenever you need a little unpredictability in your code. It's like having a surprise button! You can use it for:

- **Games:** Want to roll dice, shuffle cards, or make a character appear in a random spot? The random module makes it happen!
- **Choosing Random Items:** Need to pick a random winner for a giveaway or choose a random question from a quiz? This module has you covered.
- **Simulations:** Sometimes, we want to mimic real-world scenarios where things happen by chance, like flipping a coin or predicting weather patterns. The random module helps create that "real-life" randomness.
- **Generating Random Data:** Whether it's a random password, a number for testing, or even random colors for an art project, this module does it all.

1.3 Tips & Tricks for the random Module

1.3.1 1. random.seed() for Repeatable Results

If you need the same “random” results every time (for testing), use **random.seed()** . This makes the random output predictable.

```
[35]: import random
      random.seed(1) # Always gives the same result
      print(random.randint(1, 10))
```

3

1.3.2 2. random.uniform() for Decimals

Need a random decimal between two numbers? Use random.uniform()!

```
[40]: import random
      print(random.uniform(1, 5)) # Random float between 1 and 5
```

4.043849779650302

1.3.3 3. random.sample() to Pick Multiple Items

Want to pick multiple random items without repeating? Use random.sample().

```
[43]: import random
      names = ['Alice', 'Bob', 'Charlie']
      print(random.sample(names, 2)) # Pick 2 random names
```

['Bob', 'Alice']

1.4 Basic Functions of the random Module

1.4.1 1. random.randint(a, b)

This gives you a random number between a and b (including both). It’s like rolling a dice where you can choose the range of numbers.

```
[44]: import random
      # Get a random number between 1 and 6
      dice_roll = random.randint(1, 6)
      print(dice_roll)
```

4

1.4.2 2. random.choice (list)

Want to randomly pick an item from a list? This function does exactly that! It’s great for picking a random winner or choosing a random option.

```
[25]: import random
names = ['Alice', 'Bob', 'Charlie'] # Randomly pick a name
random_name = random.choice(names)
print(random_name)
```

Bob

1.4.3 3. random.random()

This gives you a random float (decimal number) between 0 and 1. It's like rolling a super tiny dice where you always get a number between 0.0 and 1.0.

```
[45]: import random
# Get a random decimal between 0 and 1
random_float = random.random( )
print(random_float )
```

0.43276706790505337

1.4.4 4. random.shuffle (list)

Want to mix things up? This function shuffles the items in a list, changing the order around randomly. Perfect for shuffling a deck of cards or rearranging a list.

```
[31]: cards = ['Ace', 'King', 'Queen', 'Jack']
# Shuffle the List
random.shuffle(cards )
print(cards )
```

['Queen', 'Jack', 'King', 'Ace']

1.4.5 5. Examples

5.1. a simple number guessing game In the following example we will create a simple number guessing game using the random module. The computer will pick a random number between 0 and 99, and you will have 5 chances to guess it. Good luck!

```
[11]: import random
print("Hi welcome to the game, This is a number guessing game.")
print("You got 5 chances to guess the number. Let's start the game")
random_number = random.randrange(100)
chances = 5
guess_counter = 0
while guess_counter < chances:
    guess_counter = guess_counter + 1
    my_guess = int(input('Please Enter your Guess : '))
    if my_guess == random_number:
        print(f'The number is {random_number} and you found it right in_
→{guess_counter} attempt(s) !')
        break
```

```

elif guess_counter >= chances and my_guess != random_number:
    print(f'Oops sorry, The number is {random_number} better luck next time.
↪')
elif my_guess > random_number:
    print(f'Your guess is: {my_guess}, your guess is higher than the number.
↪')
elif my_guess < random_number:
    print(f'Your guess is: {my_guess}, your guess is lower than the number.
↪')

```

Hi welcome to the game, This is a number guessing game.
 You got 5 chances to guess the number. Let's start the game
 Your guess is: 50, your guess is higher than the number.
 Your guess is: 30, your guess is higher than the number.
 Your guess is: 10, your guess is higher than the number.
 Your guess is: 5, your guess is higher than the number.
 Oops sorry, The number is 0 better luck next time.

5.2. Examples of using random module

```

[24]: import random
print(random.random())           # Random float: 0.0 <= n < 1.0
print(random.randrange(15))      # Random integer from 0 to 14
print(random.randint(1, 10))     # Random integer from 1 to 10
↪(inclusive)
print(random.choice(['apple', 'banana', 'cherry'])) # Randomly select an
↪element from a list
print(random.sample(range(100), 10)) # Randomly select 10 unique numbers
↪from 0 to 99
print(random.shuffle([1, 2, 3, 4, 5])) # Shuffle a list in place, gives None
↪but the list is shuffled
print(random.uniform(1, 10))      # Random float: 1.0 <= n < 10.0
print(random.gauss(0, 1))          # Random float: Normal distribution
print(random.betavariate(1, 1))    # Random float: Beta distribution
print(random.expovariate(1))       # Random float: Exponential
↪distribution
print(random.gammavariate(1, 1))   # Random float: Gamma distribution
print(random.lognormvariate(1, 1)) # Random float: Log-normal
↪distribution
print(random.normalvariate(1, 1))  # Random float: Normal distribution
print(random.vonmisesvariate(1, 1)) # Random float: Von Mises distribution
print(random.paretovariate(1))     # Random float: Pareto distribution
print(random.weibullvariate(1, 1)) # Random float: Weibull distribution
print(random.triangular(1, 10, 5)) # Random float: Triangular
↪distribution

```

0.7692399727391376

6

```
4
cherry
[55, 61, 59, 75, 28, 70, 81, 89, 77, 23]
None
4.7705318386012525
-0.2653805448611613
0.04149230578795307
1.1372208695710295
1.828545302009997
2.7682210991018343
0.5588659463563228
4.975064917311508
1.3169818668376212
1.8044662623157683
5.151399445618591
```

1.4.6 random vs secrets for generating random numbers

secrets module is used for cryptographic purposes and is more secure than *random* module.
random module is used for general purposes and is not suitable for cryptographic use.

```
[19]: import secrets
secure_random = secrets.SystemRandom()
random_number = secure_random.randrange(100)
print(random_number)
```

26